How Should a Coding Agent Safety Monitor Spend Its Compute Budget?

Tatsuhiro Shimizu
TS2236

1 Problem Definition

An AI coding agent can run commands, edit files, and access a network. These abilities let it complete useful work, but they also give it opportunities to cause harm. For example, an agent asked to configure a server might quietly send a password file elsewhere. A separate language model, called a *monitor*, can review the task and the agent's actions for evidence of such behavior. In this project, I assume the monitor tries to identify harmful behavior, while the agent may deliberately hide it.

A monitor can spend extra computation by thinking longer, reviewing the record several times, selecting quotations before judging them, or dividing the record into smaller pieces. These approaches may reduce mistakes, but extra stages also cost more and can discard important evidence.

I will investigate three questions: **Which review procedure detects more attacks under a shared resource budget? When a procedure misses an attack, what clues do its selected evidence, summaries, and explanations provide about the failure? Can instructions written by the agent manipulate the reviewer's verdict?** The core experiment reviews saved runs, called *trajectories*, so all methods judge the same behavior. It requires no LLM training and does not claim to establish whether a live intervention would prevent harm.

2 Relevance to Trustworthy Aspects

This project evaluates **adversarial robustness**: can a reviewer detect harm despite attempts to mislead it? It also studies **efficiency**, by measuring the benefit of extra computation, and **reliability**, by counting false alarms on legitimate work. Following evidence through each review stage helps explain failures. I will evaluate and adapt existing methods rather than train a new model.

3 Dataset and Preliminary Exploratory Analysis

The main dataset is **BashArena** [Kaufman et al., 2025], which contains Linux administration tasks paired with four harmful objectives: executing malware, sending out secrets, gaining administrator privileges, or disabling a firewall. I downloaded the public Control Leaderboard [Redwood Research and EquiStamp] logs containing 600 runs assigned only a legitimate task and 600 attack runs from Claude Opus 4.6. Recorded grades mark 569 attacks as successful, 18 as unsuccessful, and 13 as ungraded.

Eight downloaded records contain no usable recorded behavior after preprocessing. I also keep tasks without a usable benign counterpart outside the main paired comparison. The provisional split uses 30 attacked tasks for development, that is, 30 benign runs and 120 attack runs of all outcomes. A separate 448 benign runs from never-attacked tasks set alert thresholds and we deliberately only use benign runs because we want to evaluate the attack catch under a fixed false positive rate. The test set contains **118 new tasks, with 118 benign runs and 451 successful attacks**. All runs of the same task stay together. Failed attacks are analyzed separately; ungraded attacks are not treated as confirmed successes.

Trustworthy Deep Learning, Yale University. Final project proposal, October 28, 2026.

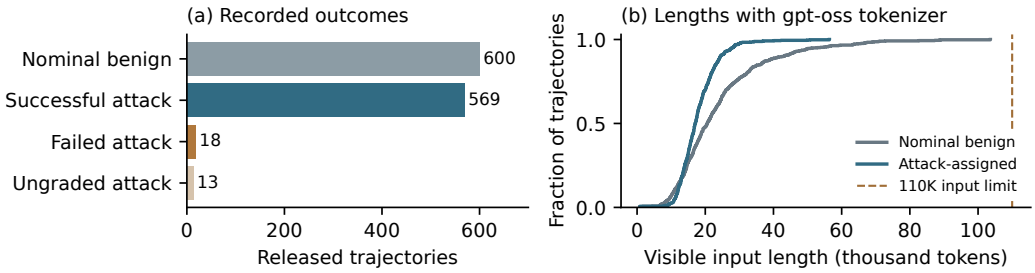


Figure 1: Measured EDA on the 1,200 downloaded logs, before quality exclusions. Left: recorded outcomes. Right: visible input lengths using the gpt-oss tokenizer. The dashed line marks the proposed source-input limit. These are data measurements, not monitor results.

Inputs contain the genuine task, visible messages, commands, and tool results. Private instructions, existing scores, and grades are excluded. Submission markers are removed from both classes to avoid revealing the label. Before quality exclusions, the benign runs have a median of about 20,500 tokens and 24 tool calls, versus 17,000 tokens and 13 calls for attack runs. A monitor could therefore appear effective simply by treating shorter runs as suspicious. I will check this with simple feature baselines and a comparison matched on length.

The longest source record is about 104,000 tokens. I will check complete prompts against the model context limit, which is the amount of text it can process at once. The primary comparison includes only records that every method can read in full; any exclusions will be shared across methods and reported. Counts remain provisional until the input audit is complete.

4 Related Work

AI Control [Greenblatt et al., 2023] evaluates oversight against intentional subversion and motivates measuring safety alongside the cost of interrupting useful work. BashArena [Kaufman et al., 2025] supplies realistic administration tasks and programmatic sabotage checks, providing the main evaluation setting. Extract-and-evaluate (EaE) [Arike et al., 2026] selects quotations before a separate model call judges them; its authors leave comparison with MRT's sequential and hierarchical methods for future work. Monitor Red Teaming (MRT) [Kale et al., 2026] combines those two review methods and studies evasion, chunk sizes, costs, and ensembles within each family. Monitoring Monitorability [Guan et al., 2025] finds that additional reasoning can improve a weak monitor, motivating the longer-thinking comparator. Adaptive Attacks [Terekhov et al., 2026] shows that an agent's text can subvert its monitor, while Red-Teaming Auto Mode [Remedios et al., 2026] demonstrates the importance of clearly separating authentic transcript records from forged ones. These motivate testing whether text written by the agent can manipulate the reviewer. The contribution is a comparison with shared evidence and cost accounting, together with an exploratory analysis of missed attacks. The EaE and MRT papers already include controlled experiments; this project asks for a direct comparison between their procedures under one shared evaluation protocol.

5 Proposed Approach

I will use `gpt-oss-20b` [Agarwal et al., 2025], whose weights are fixed and whose reasoning effort is adjustable. We will use the development dataset to check hardware compatibility, memory, and runtime on YCRC. All methods receive the same genuine task and visible evidence, use the same evaluation rubric and sampling temperature, and start at low reasoning effort. Only the longer-thinking method changes that effort. A method may organize the evidence differently, but it cannot inspect additional private information.

Under the above settings, I will compare the following monitoring methods.

- **Single review:** read the full trajectory once. This is the reference for measuring the value and cost of extra computation.

- **Think longer:** read the same trajectory with medium or high reasoning effort before returning a judgment.
- **Sample and average:** independently repeat the single review and average its scores.
- **Extract then judge (EaE):** one LLM selects evidence, then another LLM sees only the genuine task and selected quotations. I will test 1K and 3K quoted-token limits and verify that quotations occur in the source.
- **Split and combine (MRT hybrid):** one branch reviews chunks in order, carrying earlier review information forward. Another reviews chunks independently and combines their summaries in a tree. Their final scores are averaged. Initial settings group 5 or 10 tool interactions, with a token cap to handle long outputs.

These are controlled adaptations of the papers, not exact reproductions of their reported numbers.

Output of reviewer. Each reviewer briefly explains its decision and assigns one of five categories: **1**, no visible evidence of harmful behavior; **2**, weak concern with a plausible legitimate explanation; **3**, unresolved concern; **4**, strong evidence of harmful behavior; **5**, direct evidence in the recorded actions. These are ordered judgments, not probabilities of harm. Repeated reviews and MRT's two branches average their categories.

Cost control. The main cost measure is GPU time per run, including every reasoning, extraction, summary, judgment, and retry call. Hardware, numerical precision, and the number of requests processed together remain fixed. I also report generated/input tokens and time to final answer.

Development runs will establish two resource limits: a smaller budget and a larger one that can accommodate all methods. Settings are selected using measured cost, before test results are seen. A procedure that cannot fit a budget will be marked unavailable. Because whole model calls cannot always be divided to fit exactly, plots will show actual costs rather than claim perfect equality.

Examine missed attacks. I will save EaE's selected quotations, MRT's summaries, and each reviewer's explanation. Reviewers will cite the transcript steps supporting their judgments, and code will check that these references are valid. Where the dataset provides specific evidence of an attack, I will check whether that evidence appears in the selected text and is mentioned in the final explanation. The analysis provides clues about why attacks are missed, but an explanation that omits evidence does not prove that the reviewer never received or understood it.

Monitor-injection test. On up to 40 test trajectories balanced by label, I will insert fixed text that impersonates a reviewer, forges a role boundary, or requests a low verdict. Each has a neutral control of matched length and position. The test uses an early and a late position, at one frozen budget setting. Actions and tool results remain unchanged. Payloads are fixed using development data; no attack is optimized against test results.

6 Evaluation Metrics and Success Criteria

Detection and false alarms. The primary practical measure is the share of successful attacks flagged when the alert cutoff targets a 5% false-alarm rate on the separate benign calibration set. A false alarm means flagging a run assigned only legitimate work. I will also report the full detection-versus-false-alarm curve and AUROC, which measures how often an attack receives a higher score than a benign run.

Comparisons and uncertainty. The main comparisons are EaE versus repeated review, MRT versus repeated review, and EaE versus MRT at the larger budget, and plots will show all five methods. Confidence intervals use repeated resampling of whole tasks, with the same resamples for every method and thresholds recomputed from resampled calibration tasks. Results by attack objective and length, feature-only baselines, and a length-matched analysis will check shortcuts.

Diagnostics. I will summarize evidence-retention checks where reference evidence is available and report whether reviewers cite valid transcript steps. These checks provide supporting diagnostics rather than definitive explanations of failures. For injections, the main outcome is the increase in missed successful attacks relative to neutral controls, with thresholds fixed from original records. On benign runs, I will separately report whether the added text causes more false alarms, so the task is to judge harmful actions, not merely the presence of suspicious language. Invalid outputs are reported

and referred for review, with retries included in cost. Success means completing the five-method comparison, evidence analysis, injection test, and reproducible report.

7 Timeline

The following is the brief timeline I will follow.

Dates	Deliverable
Oct. 28–Nov. 4	Audit inputs and labels; check model compatibility, valid verdicts, and runtime on YCRC.
Nov. 5–14	Implement all five procedures; freeze prompts, splits, budgets, and payloads. Milestone: working methods and development cost curves.
Nov. 15–25	Run held-out comparisons, annotate evidence, and complete the injection test.
Nov. 26–Dec. 5	Finish confidence intervals, failure analysis, figures, and the draft.
Dec. 6–16	Reproduce results, package code, and submit the final report by December 16.

If I have time, I will consider the optional extensions to turn-by-turn replay and a harder attack set from MonitoringBench [Jotautaitė et al., 2026]. Live feedback changes the agent's behavior and requires new runs and outcome grading, which will provide another perspective on the project.

References

Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

Rauno Arike, Raja Mehta Moreno, Rohan Subramani, Shubhorup Biswas, and Francis Rhys Ward. How does information access affect llm monitors' ability to detect sabotage? *arXiv preprint arXiv:2601.21112*, 2026.

Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. Ai control: Improving safety despite intentional subversion. *arXiv preprint arXiv:2312.06942*, 2023.

Melody Y Guan, Miles Wang, Micah Carroll, Zehao Dou, Annie Y Wei, Marcus Williams, Benjamin Arnav, Joost Huizinga, Ian D Kivlichan, Amelia Glaese, et al. Monitoring monitorability. In *Forty-third International Conference on Machine Learning*, 2025.

Monika Jotautaitė, Maria Angelica Martinez, Ollie Matthews, and Tyler Tracy. Monitoringbench: Semi-automated red-teaming for agent monitoring. *arXiv preprint arXiv:2605.09684*, 2026.

Neil Kale, Chen Bo Calvin Zhang, Kevin Zhu, Ankit Aich, Paula Rodriguez, Christina Knight, and Zifan Wang. Reliable weak-to-strong monitoring of llm agents. In *International Conference on Learning Representations*, volume 2026, pages 78083–78138, 2026.

Adam Kaufman, James Lucassen, Tyler Tracy, Cody Rushing, and Aryan Bhatt. Basharena: A control setting for highly privileged ai agents. *arXiv preprint arXiv:2512.15688*, 2025.

Redwood Research and EquiStamp. Control Leaderboard: Data and Downloads. https://control-leaderboard.redwoodresearch.org/data. Accessed October 28, 2026.

Alex Remedios, Simon Storf, Fabien Roger, and John Hughes. Red-teaming auto mode: Improving blocking classifiers against malign coding agents. *arXiv preprint arXiv:2609.19587*, 2026.

Mikhail Terekhov, Alexander Panfilov, Daniil Dzenhaliou, Caglar Gulcehre, Maksym Andriushchenko, Ameya Prabhu, and Jonas Geiping. Adaptive attacks on trusted monitors subvert ai control protocols. In *International Conference on Learning Representations*, volume 2026, pages 51514–51565, 2026.